the identity is not being used to authorize the action in this case, because no permissions are required to create a new social space. Instead authentication is required purely for accountability; so that there is a record in the audit log of who created the space.

9.2.3 HATEOAS

You now have a capability URI returned from creating a social space, but you can't do much with it. The problem is that this URI allows access to only the resource representing the space itself, but to read or post messages to the space the client needs to access the sub-resource `/spaces/1/messages` instead. Previously, this wouldn't be a problem because the client could just construct the path to get to the messages and use the same token to also access that resource. But a capability token gives access to only a single specific resource, following POLA. To access the messages, you'll need a different capability, but capabilities are unforgeable so you can't just create one! It seems like this capability-based security model is a real pain to use.

If you are a RESTful design aficionado, you may know that having the client just know that it needs to add /messages to the end of a URI to access the messages is a violation of a central REST principle, which is that client interactions should be driven by hypertext (links). Rather than a client needing to have specific knowledge about how to access resources in your API, the server should instead tell the client where resources are and how to access them. This principle is given the snappy title *Hypertext as the Engine of Application State*, or HATEOAS for short. Roy Fielding, the